Alexandre Schneider 6844 FDU MSACS

GitHub Repo: GitHub - alxschneider/DistributedDataAccessLab

This report documents the second deliverable (V2) of the DistributedDataAccessLab project. It extends the midterm containerized e-commerce marketplace backend by adding asynchronous messaging with RabbitMQ, an API Gateway using YARP, and DTO-based design across all services.

Summary of Improvements from Midterm

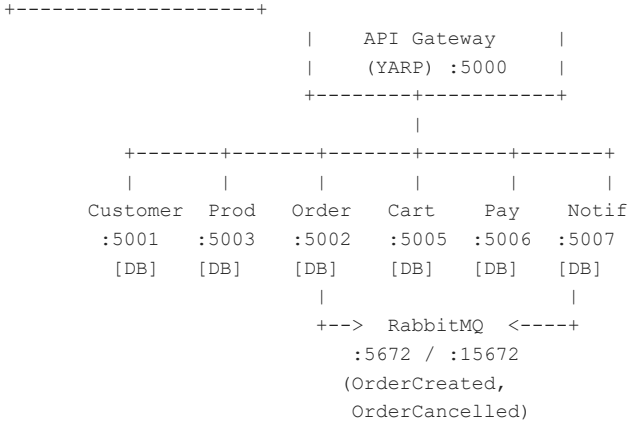
- RabbitMQ messaging via MassTransit: OrderCreatedEvent and OrderCancelledEvent
- API Gateway (YARP) as single entry point with route mapping for all 6 services
- Aggregated endpoint combining Order + Customer + Product data in a single call
- DTO records for all request/response models across every service
- Updated docker-compose.yml with RabbitMQ container and API Gateway service

Key Points

- 7 APIs: Customer, Product, Order, Cart, Payment, Notification, API Gateway
- Each service has its own SQLite database — no shared data store
- Everything runs inside Docker containers via docker compose up --build
- Async inter-service communication via RabbitMQ (MassTransit)
- Synchronous inter-service calls via typed HttpClient
- All APIs use DTOs — domain entities are never exposed directly

Architecture Overview

The API Gateway (YARP) on port 5000 is the single entry point. It routes requests to 6 microservices: Customer (:5001), Product (:5003), Order (:5002), Cart (:5005), Payment (:5006), and Notification (:5007). Each service has its own SQLite database. The Order Service publishes events to RabbitMQ, and the Notification Service consumes them. Cross-service data access happens through HTTP REST calls or async events — no service references another's code or database.



API Gateway Design

Routing

YARP is configured in appsettings.json with route-to-cluster mappings. Each /api/* path is forwarded to the corresponding microservice inside Docker. The original path is preserved.

- /api/customers/* -> customerservice:8080
- /api/products/* -> productservice:8080
- /api/orders/* -> orderservice:8080
- /api/cart/* -> cartservice:8080
- /api/payments/* -> paymentservice:8080
- /api/notifications/* -> notificationservice:8080

Aggregated Endpoint

GET /api/aggregate/order-details/{orderId} fetches the order, then calls Customer and Product services in parallel (Task.WhenAll), returning a combined { Order, Customer, Products } response. This reduces multiple client round-trips to a single call.

Messaging Design (RabbitMQ)

MassTransit is used over RabbitMQ for async communication. The Order Service publishes events; the Notification Service consumes them.

Events

- OrderCreatedEvent — published when a new order is created (fields: OrderId, BuyerId, Total, CreatedAt, Items[])
- OrderCancelledEvent — published when an order is cancelled (fields: OrderId, BuyerId, Reason, CancelledAt, Items[])

Producer: Order Service

OrdersController injects IPublishEndpoint. After saving an order, it publishes OrderCreatedEvent. After cancelling, it publishes OrderCancelledEvent.

Consumers: Notification Service

- OrderCreatedConsumer — creates notifications for the buyer and each seller involved
- OrderCancelledConsumer — notifies buyer and sellers about cancellation

MassTransit auto-creates queues named after the consumer classes. Both services connect to RabbitMQ using the Docker hostname 'rabbitmq' with guest/guest credentials.

DTO Design

All services use DTOs (C# records) for request and response models. Domain entities are never exposed directly. Each controller has a MapToDto() method for the conversion. This decouples API contracts from the database schema and prevents leaking internal fields.

- CustomerService — CreateCustomerDto, UpdateCustomerDto, CustomerResponseDto, CustomerDetailDto, etc.
- ProductService — CreateProductDto, UpdateProductDto, ProductResponseDto, ProductDashboardDto, etc.
- OrderService — CreateOrderRequestDto, OrderResponseDto, OrderDashboardDto, BuyerDashboardDto, etc.
- CartService — AddCartItemDto, CartResponseDto, CartItemResponseDto, CheckoutResponseDto
- PaymentService — CreatePaymentDto, PaymentResponseDto, PaymentDashboardDto, etc.
- NotificationService — CreateNotificationDto, NotificationResponseDto, NotificationPageDto, etc.

Docker & Containerization

All services use multi-stage Dockerfiles. The system starts with docker compose up --build. The docker-compose.yml defines:

- RabbitMQ (:5672/:15672) with healthcheck
- 6 microservices, each with its own SQLite database persisted via volume mounts
- API Gateway (:5000) depending on all services

Service discovery uses Docker Compose DNS. URLs are injected via environment variables (e.g. Services__OrderService: http://orderservice:8080/). Each service auto-migrates its database on startup.

Challenges & Solutions

- Shared contracts without shared library: MassTransit routes by namespace, so both services duplicate the contract records with the same namespace (Contracts.Events).
- Container startup order: RabbitMQ has a healthcheck; dependent services use depends_on with condition: service_healthy.
- YARP environment overrides: cluster addresses are overridden in docker-compose.yml via .NET's double-underscore convention (ReverseProxy__Clusters__...__Address).

Conclusion

This deliverable extends the midterm with RabbitMQ messaging, YARP API Gateway, and DTOs across all services. The full system runs with a single docker compose up --build command. All services maintain separate databases, communicate through APIs and events, and never expose domain entities directly.